

武汉理工大学

《企业级应用软件设计与开发》课程报告

ResearchFlow-Agent

面向研究生的自动化文献调研与报告生成智能体

学 院	计算机与人工智能学院
专 业	软件工程
学 号	2025303010
姓 名	万骁然
方 向	Agentic AI 原生开发
指导教师	戚欣

2026 年 6 月 22 日

摘要

ResearchFlow-Agent 是一个面向研究生文献调研场景的自动化研究智能体 Demo。项目围绕“用户输入研究主题后，系统自动检索论文、筛选代表性论文、生成中文结构化总结并输出 Markdown 报告”的闭环目标展开。当前版本使用 Streamlit 提供交互页面，使用 LangGraph 编排 Planner、Search、Filter、Summary、Report、Evaluator 多步骤工作流，使用 OpenAlex 和 arXiv 作为论文检索工具，并使用 DeepSeek API 生成论文总结；在缺少密钥、网络失败或 dry-run 模式下，系统会自动 fallback 到 mock 总结或 demo 论文，保证课程演示可稳定运行。

项目重点体现了 Agentic AI 原生开发中的工具调用、状态管理、多步骤推理、可观测性、缓存、成本保护、环境变量安全和 benchmark 评估。论文筛选阶段采用可解释规则评分： $\text{relevance_score} = \text{keyword_score} + \text{year_score} + \text{citation_score}$ ，其中关键词评分进一步考虑关键词覆盖率、标题命中比例和摘要命中比例，引用评分采用年均引用量，以降低老论文总引用量带来的偏差。当前项目还没有实现 MCP、Function Calling、向量数据库、全文 PDF RAG 和云部署，这些内容作为后续扩展方向保留。

目录

摘要	2
目录	3
一、选题背景与设计思想	5
1.1 选题背景	5
1.2 为什么适合 Agentic AI	5
1.3 项目价值	5
1.4 技术路线	5
二、Specs 规格文档	6
2.1 Product Spec	6
2.2 Architecture Spec	7
2.3 API / Interface Spec	8
三、系统架构与设计	9
3.1 总体架构	9
3.2 Agent 工作流图	10
3.3 数据流设计	10
3.4 工程边界	11
四、关键实现与代码展示	11
4.1 LangGraph 串联多步骤 Agent 节点	11
4.2 Search Node 的工具调用	11
4.3 OpenAlex 和 arXiv 工具层	12
4.4 可解释论文筛选公式	12
4.5 DeepSeek 总结与 Mock fallback	13
4.6 报告生成与输出分离	13
4.7 Streamlit 页面展示	13
4.8 缓存与历史记录	14
五、测试与评估	14

5.1 功能测试	14
5.2 Agent 行为评估	14
5.3 Benchmark 评估	15
5.4 Demo 截图展示	15
六、系统升级与扩展	19
6.1 已完成版本演进	19
6.2 后续扩展方向	19
七、课程总结	20
参考资料	20
附录 A：运行命令	21

一、选题背景与设计思想

1.1 选题背景

研究生在开始一个新研究方向时，通常需要快速了解该方向的代表性论文、主要方法、研究问题和已有局限。传统文献调研通常依赖手工搜索数据库、逐篇阅读摘要、整理笔记和撰写综述报告。这个过程存在几个明显痛点：

- 检索成本高：同一主题可能需要在多个学术平台反复搜索，手动去重和筛选效率较低。
- 筛选标准不清晰：初学者容易只看标题或引用量，忽视主题相关性、年份、摘要信息等因素。
- 总结效率低：英文论文摘要需要逐篇理解，再转换成中文结构化报告。
- 报告结构不稳定：不同用户写出的文献调研报告格式不统一，难以直接用于课程展示或后续研究计划。
- API 和 LLM 调用存在不确定性：网络失败、API 限流、LLM 费用和密钥安全都需要工程化处理。

因此，本项目选择构建一个轻量级 **Research Agent Demo**，让用户只需要输入研究主题，例如 **Agentic RAG**，系统就能自动完成论文检索、筛选、总结和报告生成。

1.2 为什么适合 Agentic AI

文献调研不是一次单纯的文本生成任务，而是由多个步骤组成的流程型任务：理解主题、规划关键词、调用检索工具、筛选候选论文、调用 LLM 总结、生成报告、保存输出并评估结果。这种任务天然适合 **Agentic AI** 的思想，即让系统围绕目标自主调用工具、维护中间状态，并按照多步骤流程完成任务。

ResearchFlow-Agent 并不声称已经达到生产级 **DeepResearch** 或 **GPT Researcher** 的能力。它的定位是课程项目中的最小可运行研究智能体，重点展示 **Agentic AI** 原生开发的核心结构： workflow 编排、工具调用、状态流转、可恢复 **fallback**、可解释评分和 **Demo** 可观测性。

1.3 项目价值

本项目的价值主要体现在三个方面：

1. 对用户：降低初步文献调研门槛，快速得到结构化中文报告。
2. 对课程：覆盖 **Agentic AI** 开发中的工具调用、状态管理、评估、安全配置等核心要求。
3. 对后续扩展：代码结构按模块拆分，后续可以继续升级为更复杂的 **LangGraph** 多分支 workflow、**Function Calling**、**MCP** 或全文 **RAG** 系统。

1.4 技术路线

项目采用如下技术路线：

用户输入 topic

- > Planner 生成检索关键词
- > Search 调用 OpenAlex / arXiv / demo fallback
- > Filter 使用可解释规则筛选代表性论文
- > Summary 使用 DeepSeek API 或 mock fallback 总结
- > Report 生成 Markdown 文献调研报告
- > Evaluator 生成轻量评估指标
- > History / Outputs 保存运行结果

二、Specs 规格文档

本节将 Product Spec、Architecture Spec 和 API / Interface Spec 集中写入最终报告中，作为当前阶段的 SDD 规格驱动开发材料。仓库中已有 docs/proposal.md，但目前还没有独立的 product_spec.md、architecture.md、api_spec.md 和 evaluation.md。因此，当前 SDD 属于“部分实现”：项目有规格意识和模块化接口，但正式规格文档仍可继续补齐。

2.1 Product Spec

2.1.1 目标用户

- 研究生：需要快速了解新研究方向，生成初步文献调研材料。
- 课程项目展示者：需要演示一个可运行的 Agentic AI Demo。
- 后续开发者：需要在现有基础上扩展 LangGraph、多工具、RAG 或评估模块。

2.1.2 核心需求

系统需要支持以下用户需求：

- 输入一个研究主题，例如 Agentic RAG。
- 自动检索相关论文，优先使用 OpenAlex，必要时使用 arXiv 或 demo fallback。
- 获取论文标题、作者、年份、摘要、引用量、链接、venue 和来源。
- 筛选 3 到 5 篇代表性论文。
- 基于标题和摘要生成中文结构化总结。
- 输出 Markdown 格式文献调研报告。
- 在 Streamlit 页面展示论文结果、执行日志、评估指标、历史记录和报告下载入口。

2.1.3 输入与输出

输入：

- topic: 研究主题。
- limit: 检索候选论文数量。
- top_k: 最终筛选论文数量。
- 环境变量：LLM、API、代理、dry-run、缓存相关配置。

输出：

- `src/outputs/latest_report.md`: 最近一次运行自动生成的报告。
- `src/outputs/reports/{topic}_{timestamp}.md`: 历史归档报告。
- `src/outputs/eval_results.md`: benchmark 评估结果。
- `src/outputs/history.jsonl`: 轻量任务历史。
- Streamlit 页面中的可视化展示。

2.1.4 使用场景

典型使用场景如下：

4. 学生准备课程或开题，需要快速了解某个方向。
5. 用户输入主题并点击“开始调研”。
6. 系统自动检索论文并筛选代表性文献。
7. 用户查看论文卡片、评分原因和 Markdown 报告。
8. 用户下载报告或复制报告内容继续扩写。

2.1.5 非功能需求

- 安全性：API Key 只能保存在 `.env` 或系统环境变量，不得硬编码到代码。
- 可维护性：代码按 `tools`、`agents`、`workflow`、`evaluation`、`memory`、`cache` 模块拆分。
- 可演示性：即使没有 DeepSeek Key 或外部学术 API 不稳定，也能通过 `mock fallback` / `demo fallback` 跑通。
- 成本控制：支持 `LLM_DRY_RUN`、`MAX_LLM_PAPERS`、摘要截断、超时和重试限制。
- 可观测性：保留 `logs`、`evaluation`、`benchmark`、`history` 等轻量观测能力。

2.2 Architecture Spec

系统主要模块如下：

- 前端交互层：`src/app.py`，使用 Streamlit 展示输入、论文结果、执行日志、评估结果和报告。
- CLI 入口：`src/main.py`，支持命令行运行完整调研流程。
- LangGraph 工作流层：`src/workflow/research_graph.py`、`src/workflow/nodes.py`、`src/workflow/research_state.py`。
- 工具层：`src/tools/openalex_search_tool.py`、`src/tools/arxiv_search_tool.py`、`src/tools/demo_paper_tool.py`、`src/tools/http_client.py`。
- Agent 功能层：`src/agents/filter_agent.py`、`src/agents/summary_agent.py`、`src/agents/report_agent.py`。
- 评估层：`src/evaluation/metrics.py`、`src/evaluation/run_evaluation.py`、`src/evaluation/benchmark_topics.json`。
- 记忆与缓存层：`src/memory/history_store.py`、`src/cache/cache_store.py`。
- 配置层：`src/config.py`、`.env.example`。

Agent 节点职责：

节点	作用	对应文件
Planner	规范化主题并生成关键词	src/workflow/nodes.py
Search	多 query 检索论文并去重	src/workflow/nodes.py
Filter	使用可解释评分筛选代表性论文	src/agents/filter_agent.py
Summary	调用 DeepSeek 或 mock fallback 总结论文	src/agents/summary_agent.py
Report	生成并保存 Markdown 报告	src/agents/report_agent.py
Evaluator	计算轻量评估指标	src/workflow/nodes.py

2.3 API / Interface Spec

2.3.1 命令行入口

```
python src/main.py "Agentic RAG"
```

可选参数：

```
python src/main.py "Agentic RAG" --limit 10 --top-k 5
```

2.3.2 Streamlit 前端入口

```
streamlit run src/app.py
```

页面支持输入研究主题、设置 limit 和 top_k、运行完整 workflow、查看论文卡片、执行日志、Evaluation JSON、历史记录和 Markdown 报告。

2.3.3 关键环境变量

```
LLM_PROVIDER=deepseek
```

```
LLM_DRY_RUN=true
```

```
MAX_LLM_PAPERS=5
```

```
MAX_ABSTRACT_CHARS=2000
```

```
LLM_TIMEOUT_SECONDS=30
```

```
LLM_MAX_RETRIES=1
```

```
DEEPSEEK_API_KEY=
```

```
DEEPSEEK_BASE_URL=https://api.deepseek.com
```

```
DEEPSEEK_MODEL=deepseek-chat
```

```
USE_DEMO_FALLBACK=true
```

```
NETWORK_PROXY=
```

```
OPENALEX_EMAIL=
```

```
OPENALEX_BASE_URL=https://api.openalex.org/works
```

```
ARXIV_BASE_URL=http://export.arxiv.org/api/query
```

注意：.env 只允许本地使用，不得提交 GitHub。env.example 只保留变量名和示例值，不写真实密钥。

2.3.4 输出文件

文件	说明	是否建议提交
src/outputs/sample_report.md	人工确认过的稳定展示样例	可提交
src/outputs/latest_report.md	最近一次运行报告	不建议提交
src/outputs/reports/	历史归档报告	不建议提交
src/outputs/eval_results.md	benchmark 评估结果	可提交
src/outputs/history.jsonl	本地任务历史	不建议提交
src/outputs/cache/	检索和总结缓存	不建议提交

三、系统架构与设计

3.1 总体架构

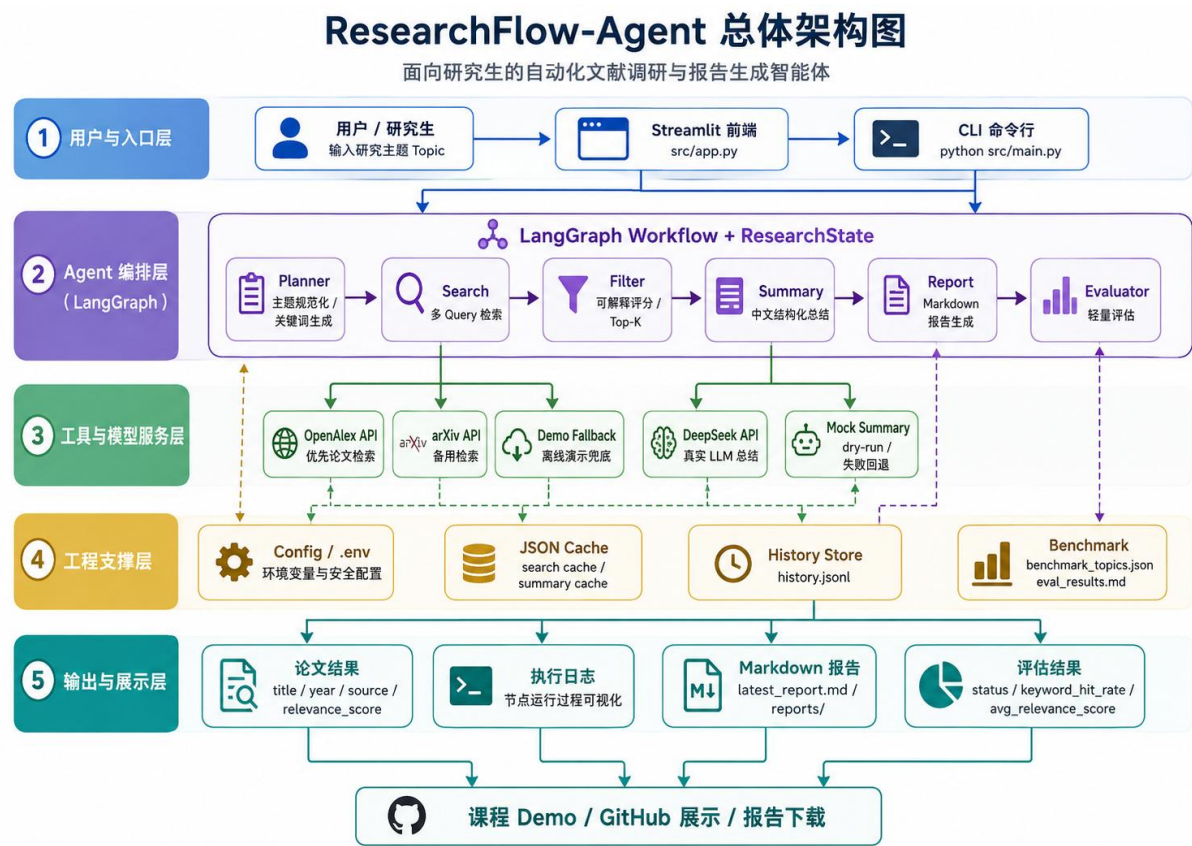


图 1 展示了 ResearchFlow-Agent 的总体架构。系统从用户输入研究主题开始，通过 Streamlit 页面或 CLI 命令行进入 LangGraph Agent 工作流。核心工作流依次完成 Planner 主题规划、Search 多 Query 检索、Filter 可解释评分筛选、Summary 中文结构化总结、Report Markdown 报告生成和 Evaluator 轻量评估。底层工具与模型服务包括 OpenAlex、arXiv、Demo fallback、DeepSeek API 和 Mock Summary；工程支撑层提供环境变量安全配置、JSON 缓存、历史记录与 benchmark 评估；最终输出论文结果、执行日志、Markdown 报告和评估结果，用于课程 Demo、GitHub 展示和报告下载。

3.2 Agent 工作流图

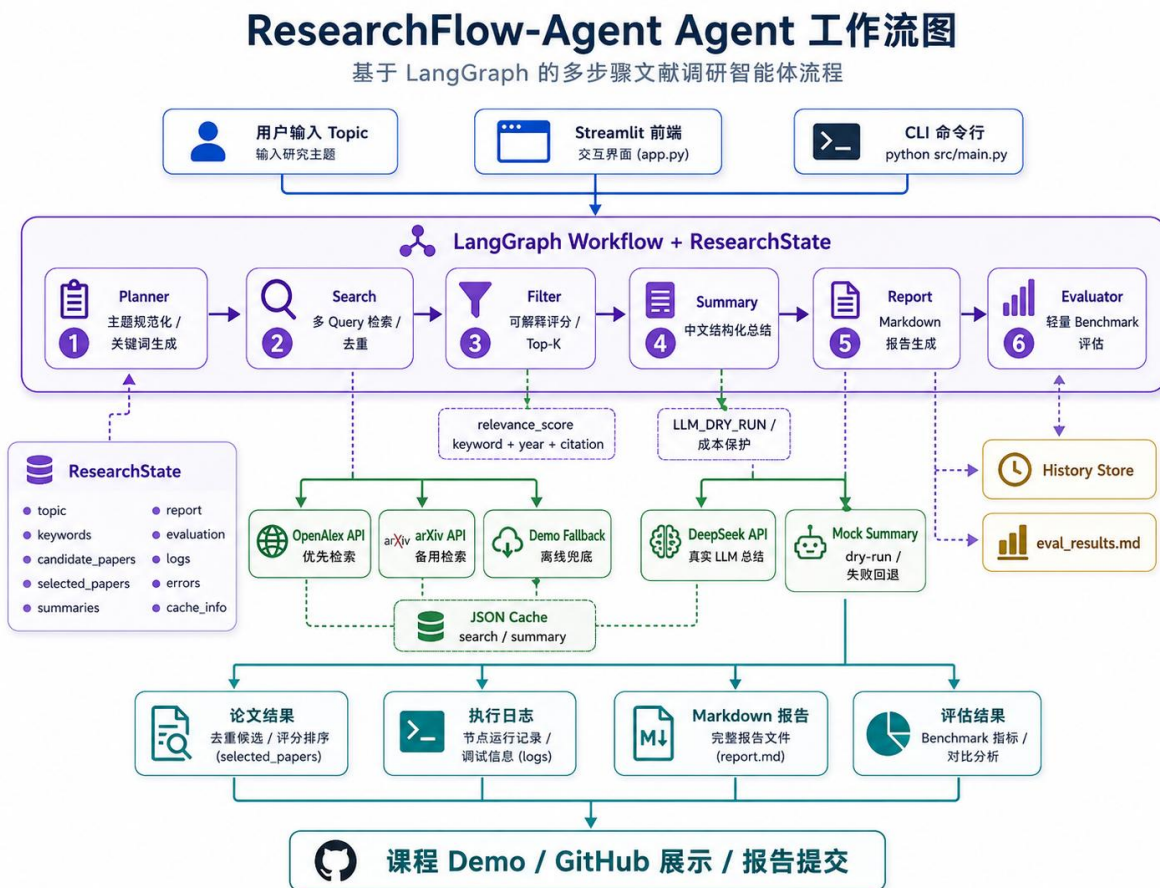


图 2 展示了 ResearchFlow-Agent 的 Agent 工作流细节。用户可以通过 Streamlit 前端或 CLI 命令行输入研究主题，系统随后进入 LangGraph Workflow。工作流以 ResearchState 作为共享状态，依次经过 Planner、Search、Filter、Summary、Report 和 Evaluator 六个节点：Planner 生成主题关键词，Search 调用 OpenAlex / arXiv 并结合 demo fallback 获取候选论文，Filter 使用可解释评分筛选 Top-K 论文，Summary 根据 dry-run、DeepSeek API 或 mock fallback 生成中文总结，Report 生成 Markdown 报告，Evaluator 输出 benchmark 评估结果。图中同时展示了 JSON Cache、History Store、执行日志、论文结果和评估结果等工程支撑模块，体现了项目的多步骤 Agent 编排、工具调用、状态管理和可观测性设计。

3.3 数据流设计

LangGraph 中的共享状态定义在 `src/workflow/research_state.py`，核心字段包括：

- `topic`、`limit`、`top_k`: 用户输入和运行参数。
- `keywords`: Planner 生成的检索关键词。
- `candidate_papers`: Search 得到的候选论文。
- `selected_papers`: Filter 筛选后的代表性论文。
- `summaries`: Summary 生成的结构化总结。

- `report`、`report_path`、`archive_report_path`: 报告内容和保存路径。
- `evaluation`: Evaluator 输出的轻量评估结果。
- `cache_info`、`logs`、`errors`: 缓存、日志和错误信息。

这种集中状态设计使每个节点只关注输入状态的一部分，同时将输出写回统一状态对象，便于 Streamlit 页面展示和 `benchmark` 评估。

3.4 工程边界

当前项目是课程 Demo，不是生产级系统。已经实现的部分包括：论文元数据检索、规则筛选、摘要级总结、Markdown 报告、Streamlit UI、缓存、历史记录和 `benchmark`。尚未实现的部分包括：PDF 全文解析、向量数据库、长期语义记忆、MCP、Function Calling、CI/CD 和云部署。

四、关键实现与代码展示

4.1 LangGraph 串联多步骤 Agent 节点

`src/workflow/research_graph.py` 使用 `StateGraph` 串联 workflow:

```
graph = StateGraph(ResearchState)
graph.add_node("planner", planner_node)
graph.add_node("search", search_node)
graph.add_node("filter", filter_node)
graph.add_node("summary", summary_node)
graph.add_node("report", report_node)
graph.add_node("evaluator", evaluator_node)

graph.set_entry_point("planner")
graph.add_edge("planner", "search")
graph.add_edge("search", "filter")
graph.add_edge("filter", "summary")
graph.add_edge("summary", "report")
graph.add_edge("report", "evaluator")
graph.add_edge("evaluator", END)
```

这段代码解决的问题是：把文献调研拆成明确的多步骤状态流，而不是在一个函数中顺序堆叠逻辑。这样后续可以继续增加条件分支、错误恢复或人工确认节点。

4.2 Search Node 的工具调用

Search Node 读取 Planner 生成的 `topic` 和 `keywords`，构造多个 `query`，优先调用 `OpenAlex`。如果 `OpenAlex` 没有结果，再尝试 `arXiv`；如果外部 API 都不可用，则使用 `demo fallback`。

```
queries = _build_search_queries(topic, state.get("keywords", []))
for query in queries:
    openalex_papers = search_openalex_papers(query, limit=per_query_limit)
    if openalex_papers:
        collected_papers.extend(openalex_papers)
        continue
```

```

arxiv_papers = search_arxiv_papers(query, limit=per_query_limit)
if arxiv_papers:
    collected_papers.extend(arxiv_papers)

```

这段实现体现了 **Tool Use: Agent** 节点不是只生成文本，而是主动调用外部学术检索工具获取结构化数据。项目还对 `title / url` 做去重，并按用户设定的 `limit` 截断候选论文数量。

4.3 OpenAlex 和 arXiv 工具层

OpenAlex 工具位于 `src/tools/openalex_search_tool.py`。它调用 OpenAlex Works API，恢复 `abstract_inverted_index`，并规范化返回字段：

```

papers.append(
    {
        "title": title,
        "authors": _extract_authors(work),
        "year": work.get("publication_year") or "N/A",
        "abstract": abstract,
        "citationCount": work.get("cited_by_count") or 0,
        "url": _extract_url(work),
        "venue": _extract_venue(work),
        "source": "OpenAlex",
    }
)

```

arXiv 工具位于 `src/tools/arxiv_search_tool.py`，使用 Python 标准库 `urllib` 调用 arXiv Atom API，并转换为同样的数据结构。统一字段后，后续 **Filter**、**Summary** 和 **Report** 不需要关心论文来自哪个平台。

4.4 可解释论文筛选公式

论文筛选逻辑位于 `src/agents/filter_agent.py`。当前评分公式为：

```
relevance_score = keyword_score + year_score + citation_score
```

其中：

```

keyword_score = 25 * keyword_coverage
                + 12 * title_hit_ratio
                + 8 * abstract_hit_ratio

```

年份分与引用分采用分段规则：

- **year_score**: 2 年以内 25 分，4 年以内 20 分，7 年以内 14 分，10 年以内 8 分，10 年以上 3 分。
- **citation_score**: 基于 $\text{citation_per_year} = \text{citationCount} / \max(1, \text{current_year} - \text{year} + 1)$ ，最高 30 分。

筛选门槛为：

```

preferred_papers:
keyword_score >= 10 或 keyword_coverage >= 0.25

```

如果没有论文满足该条件，则 **fallback** 到全部候选论文排序，避免页面空白。这个设计兼顾了主题相关性和 Demo 稳定性。

4.5 DeepSeek 总结与 Mock fallback

src/agents/summary_agent.py 使用 OpenAI-compatible SDK 调用 DeepSeek API。项目当前主 LLM 是 DeepSeek，不是 OpenAI API。OpenAI SDK 在这里只是兼容客户端。

```
client = OpenAI(  
    api_key=DEEPSEEK_API_KEY,  
    base_url=DEEPSEEK_BASE_URL,  
    timeout=LLM_TIMEOUT_SECONDS,  
)
```

为了控制成本和保证演示稳定，项目保留：

- LLM_DRY_RUN：开发阶段避免真实调用 DeepSeek。
- MAX_LLM_PAPERS：限制每次真实总结论文数量。
- MAX_ABSTRACT_CHARS：限制发送给 LLM 的摘要长度。
- LLM_TIMEOUT_SECONDS 和 LLM_MAX_RETRIES：控制超时和重试。
- mock fallback：DeepSeek 调用失败时仍能生成规则模板总结。

v0.3.8 以后项目还支持读取 .env 中的 NETWORK_PROXY，只在 DeepSeek SDK client 中使用项目级代理，不要求用户设置系统全局代理。

4.6 报告生成与输出分离

src/agents/report_agent.py 负责生成 Markdown 报告。报告包含研究主题、检索与筛选说明、代表性论文列表、单篇结构化总结、文献对比表、初步结论和参考文献。

报告保存采用分离策略：

- sample_report.md：稳定展示样例，不被普通运行覆盖。
- latest_report.md：最近一次运行自动生成的报告。
- reports/：每次运行的历史归档报告。

这个设计避免 benchmark 或普通调研覆盖人工确认过的展示样例。

4.7 Streamlit 页面展示

src/app.py 提供前端 Demo。当前 v0.3.9 页面使用 sidebar 展示运行配置、项目状态和安全提示，主区域展示主题输入、运行按钮、论文结果、Markdown 报告、执行日志与评估、历史记录。

论文结果使用 expander 展示，每篇论文包含：

- title
- year
- source
- citationCount
- relevance_score

- score_breakdown
- score_reason
- url

这使页面更适合课程展示和 GitHub 截图。

4.8 缓存与历史记录

缓存模块位于 `src/cache/cache_store.py`，使用本地 JSON 文件保存 search cache 和 summary cache。缓存 key 使用哈希生成，不保存 API Key。

历史记录模块位于 `src/memory/history_store.py`，以 JSONL 形式记录最近任务，包括 topic、候选论文数、筛选论文数、实际 LLM Provider、evaluation status、报告路径和错误类型。它是轻量任务历史，不是向量数据库，也不是长期语义记忆。

五、测试与评估

5.1 功能测试

项目支持以下基本测试：

```
python src/main.py "Agentic RAG"
streamlit run src/app.py
python src/evaluation/run_evaluation.py
```

命令行运行会输出候选论文数量、筛选论文数量、报告保存路径、配置 LLM Provider、实际 LLM Provider 和 evaluation status。Streamlit 页面可以交互运行完整 workflow，并展示论文结果、执行日志、评估结果、历史记录和 Markdown 报告。

5.2 Agent 行为评估

Agent 行为评估主要检查闭环是否成立：

- 1 是否能根据主题生成关键词。
- 2 是否能检索候选论文。
- 3 是否能筛选代表性论文。
- 4 是否能生成结构化总结。
- 5 是否能保存 Markdown 报告。
- 6 是否能生成 evaluation 指标。

Evaluator Node 会生成轻量规则评估，包括：

- candidate_count
- paper_count
- has_report

- `has_references`
- `has_comparison_table`
- `has_errors`
- `llm_provider`
- `fallback_used`
- `keyword_hit_rate`
- `avg_relevance_score`

5.3 Benchmark 评估

Benchmark 主题保存在 `src/evaluation/benchmark_topics.json`，当前包括：

- Agentic RAG
- Retrieval-Augmented Generation
- Multi-Agent Collaboration
- LLM-based Code Generation
- Memory-Augmented Agents

运行 `python src/evaluation/run_evaluation.py` 后，会生成 `src/outputs/eval_results.md`。当前评估结果显示：

指标	当前结果
Benchmark topics	5
Pass count	5
Pass rate	100.0%
Average runtime	10.9s
Average keyword hit rate	0.96
Average relevance score	59.48
Error count	0

这说明在当前本地环境中，系统能够跑通 5 个 benchmark 主题，并生成有效报告。不过，该评估仍然是轻量规则评估，不等同于人工学术质量评审。后续可以增加人工标注集、论文相关性人工评分和更严格的摘要质量评估。

5.4 Demo 截图展示

以下截图展示了 ResearchFlow-Agent 在本地环境中的运行页面、论文结果、报告输出和 Codex 辅助开发过程。



图 3 展示了 Streamlit 首页和运行配置。页面左侧展示当前版本、LLM Provider、dry-run 状态和 demo fallback 状态；主区域支持输入研究主题、设置检索数量和筛选数量，并启动完整调研流程。



图 4 展示了最近任务历史。系统会记录 topic、候选论文数量、筛选论文数量、实际 LLM Provider、评估状态和报告路径，便于课程演示时说明 Agent 运行过程可追踪。



图 5 展示了调研完成后的评估概览和代表性论文结果。页面显示候选论文数量、筛选论文数量、evaluation status、实际 LLM、keyword hit rate、平均相关性评分、缓存命中情况以及单篇论文的评分构成和筛选理由。

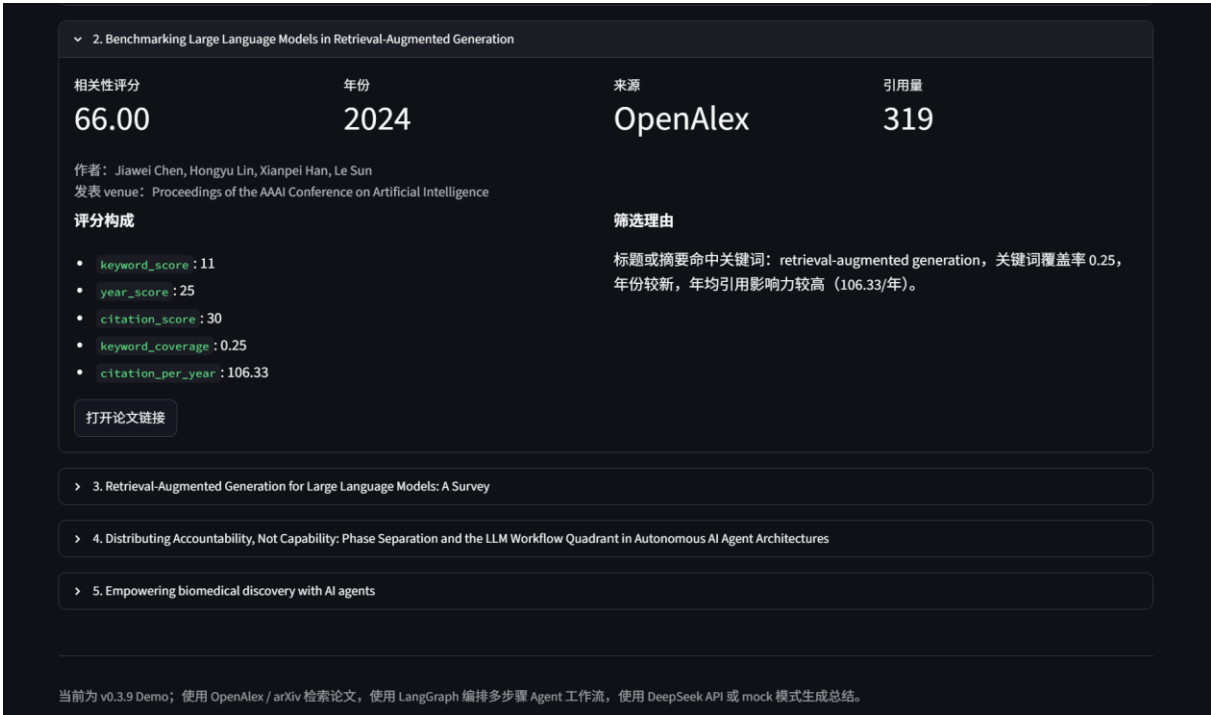


图 6 展示了更多筛选后论文条目。每篇论文都包含相关性评分、年份、来源、引用量、评分构成和筛选理由，体现了当前项目的可解释论文筛选机制。



图 7 展示了 Markdown 报告中的检索与筛选说明。报告会说明 OpenAlex / arXiv 检索、demo fallback、缺失标题或摘要过滤，以及 $\text{relevance_score} = \text{keyword_score} + \text{year_score} + \text{citation_score}$ 的评分公式。

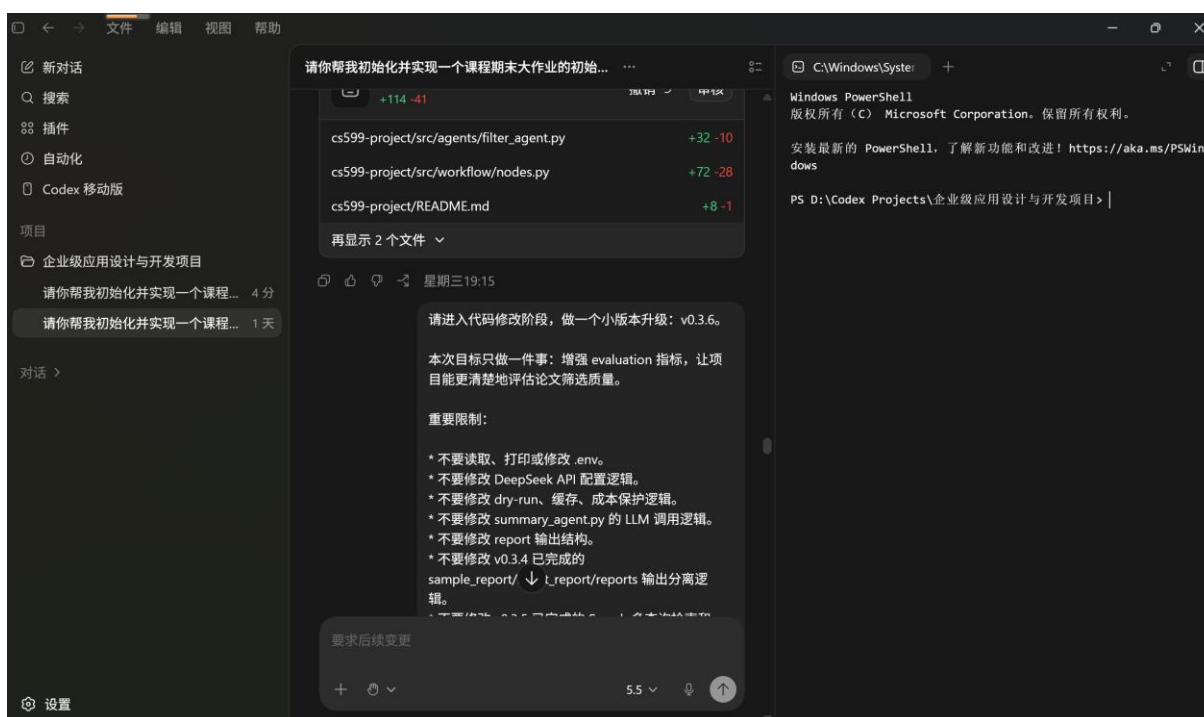


图 8 展示了使用 OpenAI Codex 辅助开发、迭代和调试项目的过程。该截图用于说明本项目在开发过程中使用了 AI Development Tool，并通过多轮任务完成版本升级。

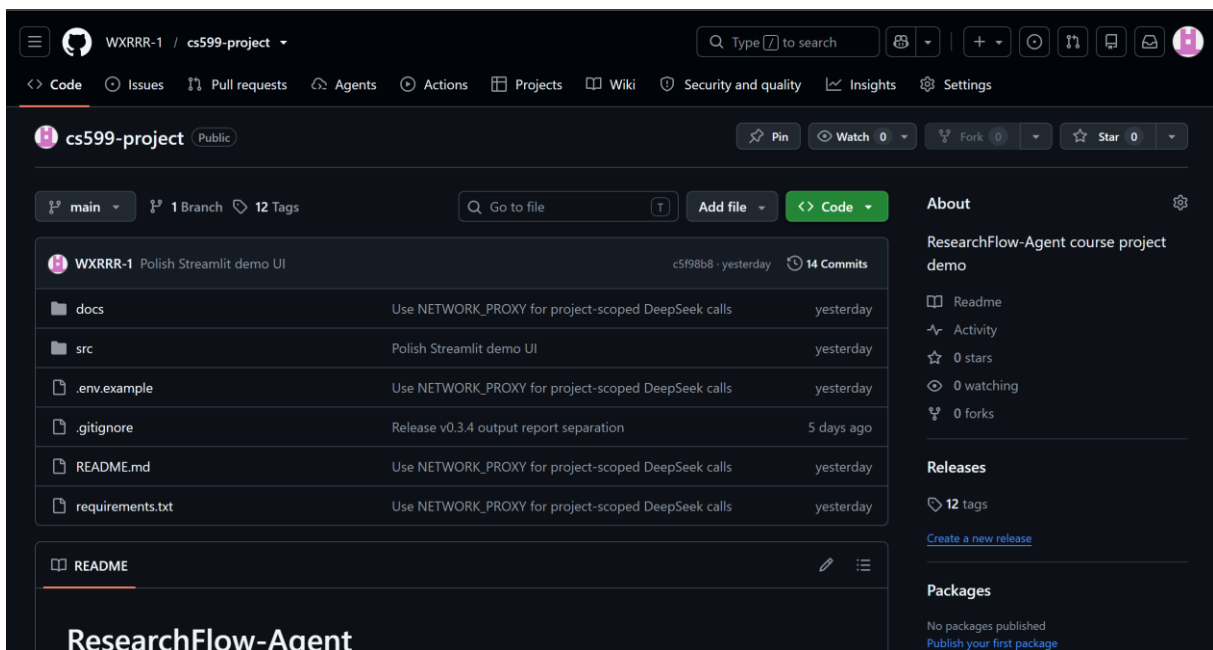


图 9 展示了 GitHub 仓库首页。仓库名为 cs599-project，包含 docs/、src/、.env.example、.gitignore、README.md 和 requirements.txt 等课程项目交付文件，并展示了提交记录、tag 数量和 README 区域。

六、系统升级与扩展

6.1 已完成版本演进

项目从 v0.1 到 v0.3.9 的演进大致如下：

- v0.1: 完成最小 Demo，支持论文检索、筛选、总结和 Markdown 报告。
- v0.2: 引入 LangGraph 工作流和 ResearchState。
- v0.3: 增加 benchmark、history、运行日志和 evaluation。
- v0.3.1: 增加 DeepSeek 成本保护、search cache 和 summary cache。
- v0.3.2: 同步报告中的评分说明。
- v0.3.4: 分离 sample_report.md、latest_report.md 和历史归档报告。
- v0.3.5: 让 Search Node 使用 topic + Planner keywords 多查询检索。
- v0.3.6: 增加 keyword_hit_rate 和 avg_relevance_score 指标。
- v0.3.7: 升级学术筛选评分，加入关键词覆盖率和年均引用量。
- v0.3.8: 支持 DeepSeek 项目级代理配置 NETWORK_PROXY。
- v0.3.9: 美化 Streamlit 前端页面，提升课程展示效果。

6.2 后续扩展方向

后续可以从以下方向扩展：

- 1 接入全文 PDF 解析：从摘要级调研升级为全文级证据抽取。
 - 2 引入向量数据库和真正的 Agentic RAG：支持论文片段检索、跨文档问答和引用定位。
 - 3 增加 Function Calling / MCP：将学术检索、报告生成、文件保存封装为标准工具接口。
 - 4 增强可观测性：引入 tracing、结构化日志、运行面板和失败重放。
 - 5 云部署：将 Streamlit Demo 部署到可访问的在线环境。
 - 6 多智能体协作：将 Planner、Reviewer、Writer、Evaluator 拆成更清晰的多 Agent 协作结构。
 - 7 更严格的评估集：构建人工标注 benchmark，对论文相关性和总结质量进行人工对照评估。
- 需要注意的是，上述内容目前大多还没有实现，不能在最终展示中写成已完成能力。

七、课程总结

通过这个项目，我对“写代码”和“设计一个 Agent 工作流”之间的区别有了更具体的理解。最开始我只是想做一个能自动查论文、生成报告的 Demo，但随着项目迭代，我逐渐意识到 Agentic AI 项目不只是调用一次大模型，而是要把任务拆成多个步骤，并让每一步都有明确输入、输出、状态和异常处理。

在开发过程中，我使用 OpenAI Codex 辅助完成项目初始化、代码重构、LangGraph 工作流升级、评估脚本、缓存和页面美化。这个过程让我体会到 AI 开发工具可以提高效率，但也需要开发者自己判断边界，例如哪些功能已经实现，哪些只是未来计划，哪些文件不能提交，哪些 API Key 不能出现在代码里。

我也开始理解 SDD 规格驱动开发的重要性。如果没有提前写清楚用户需求、输入输出、系统边界和安全约束，后续功能很容易越做越乱。这个项目虽然还不是生产级系统，但已经把论文检索、筛选、总结、报告、评估、历史记录和前端展示串成了一个可运行闭环。

此外，GitHub 版本管理也是这次课程项目中的重要收获。从 v0.1 到 v0.3.9，每个小版本都对应一次明确改进，例如 LangGraph、评估指标、筛选公式、输出分离和前端美化。通过 commit 和 tag，我可以更清楚地回看项目演进，也更容易向老师展示开发过程。

最后，我对 API Key 安全和环境变量配置有了更深认识。DeepSeek API、OpenAlex、代理配置等都不能硬编码在代码里，.env 只能本地使用，.env.example 只能放占位符。对于 Agentic AI 项目来说，功能跑通很重要，但安全、成本控制和可恢复 fallback 同样重要。

参考资料

- LangGraph 官方文档：<https://langchain-ai.github.io/langgraph/>
- LangChain 官方文档：<https://python.langchain.com/>
- Streamlit 官方文档：<https://docs.streamlit.io/>
- OpenAlex API 文档：<https://docs.openalex.org/>
- arXiv API 文档：<https://info.arxiv.org/help/api/>
- DeepSeek API 文档：<https://api-docs.deepseek.com/>

- GitHub 文档: <https://docs.github.com/>
- OpenAI-compatible Python SDK: <https://github.com/openai/openai-python>
- 本项目 GitHub 仓库: <https://github.com/WXRRR-1/cs599-project>

附录 A: 运行命令

- `cd cs599-project`
- `pip install -r requirements.txt`
- `python src/main.py "Agentic RAG"`
- `streamlit run src/app.py`
- 如果使用虚拟环境, 可以在 Windows PowerShell 中运行:
- `cd "D:\Codex Projects\企业级应用设计与开发项目\cs599-project"`
- `.\venv\Scripts\python.exe src\main.py "Agentic RAG"`
- `.\venv\Scripts\python.exe -m streamlit run src\app.py`
- Benchmark 评估命令:
- `python src/evaluation/run_evaluation.py`
- LLM 连通性检查:
- `python src/check_llm.py`